Building a POST/PATCH/DELETE API (CodeGrade)

(https://github.com/learn-co-curriculum/python-p4-building-post-patch-delete-api) (https://github.com/learn-co-curriculum/python-p4-building-post-patch-delete-api/issues/new)

Learning Goals

- Build an API to handle POST, PATCH, and DELETE requests.

Key Vocab

- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- `GET` : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- `POST` : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- `PATCH` : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- `DELETE` : an HTTP request method that signifies that the client is attempting to delete a resource.

Introduction

So far, we've seen how to set up an API with Flask to allow frontend applications to access data from a database in a JSON format. For many applications, just being able to access/read data isn't enough — what kind of app would Twitter be if you couldn't write posts? What would

Instagram be if you couldn't like photos? How embarrassing would Facebook be if you couldn't go back and delete those regrettable high school photos?

All of those applications, and most web apps, can be broadly labeled as CRUD applications — they allow users to **C**reate, **R**ead, **U**pdate, and **D**elete information.

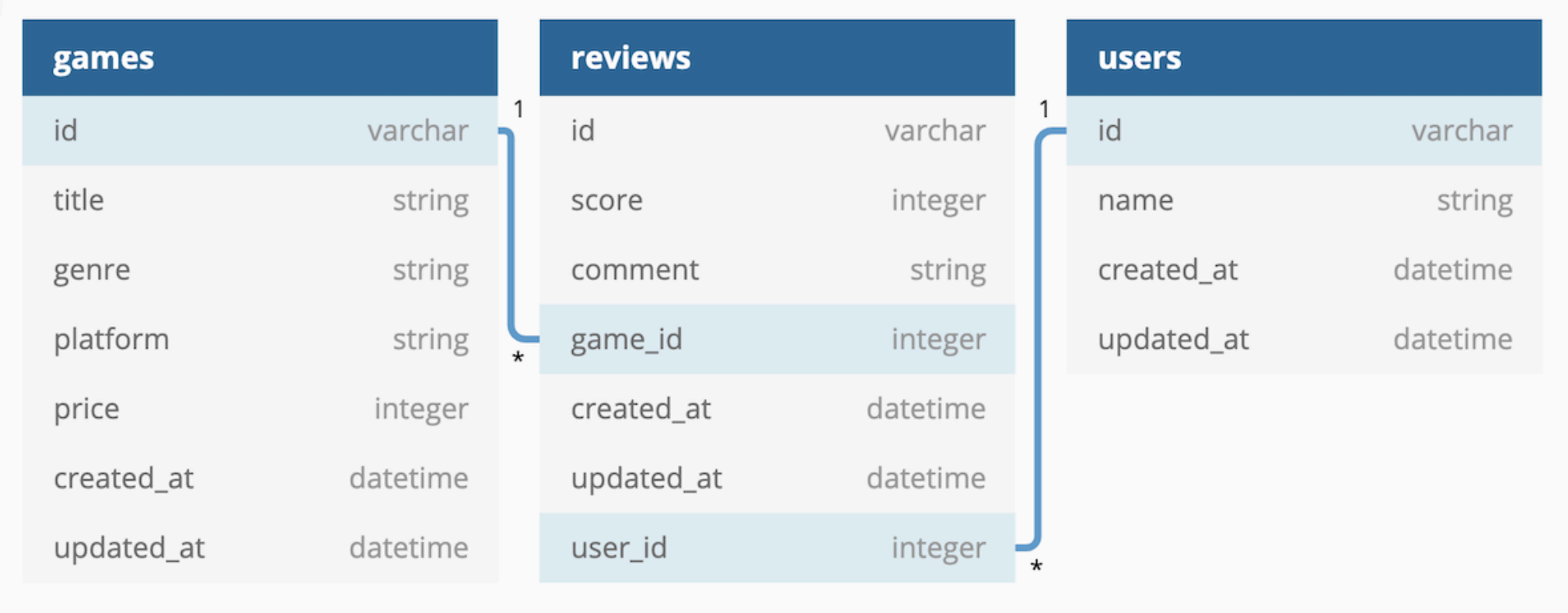
We've seen a few ways to Read data in an API. We've also already seen how to Create/Update/Delete records from a database using SQLAlchemy. All that's left is to connect what we know from SQLAlchemy with some new techniques for establishing routes and accessing data in our Flask application.

Setup

We'll continue working on the game review application from the previous lessons. To get set up, run:

```
$ pipenv install; pipenv shell
$ cd server
$ flask db upgrade
$ python seed.py
```

You can view the models in the `server/models.py` module, and the migrations in the `server/migrations/versions` directory. Here's what the relationships will look like in our ERD:



`server/app.py` has also been configured with `GET` routes for all `Review` and `User` records.

Now, run the server with Flask and re-explore some of the routes from our `GET` lesson and the new `/reviews` and `/users` routes:

```
$ python app.py
```

With that set up, let's start working on some CRUD!

Handling DELETE Requests

Let's start with the simplest action: the DELETE request. Imagine we're building a new feature in our frontend React application. Our users want some way to delete their reviews, in case they change their minds. In React, our component for handling this delete action might look

something like this:

```
// example code

function ReviewItem({ review, onDeleteReview }) {
  function handleDeleteClick() {
    fetch(`http://localhost:9292/reviews/${review.id}`, {
      method: "DELETE",
    })
      .then((r) => r.json())
      .then((deletedReview) => onDeleteReview(deletedReview));
  }

  return (
    <div>
      <p>Score: {review.score}</p>
      <p>{review.comment}</p>
      <button onClick={handleDeleteClick}>Delete Review</button>
    </div>
  );
}
```

So, it looks like our server needs to handle a few new things:

- Handle requests with the `DELETE` HTTP verb to `/reviews/<int:id>`.
- Find the review to delete using the ID.
- Delete the review from the database.
- Send a response with the deleted review as JSON to confirm that it was deleted successfully, so the frontend can show the successful deletion to the user.

Let's take things one step at a time. First, we'll need to handle requests by adding a new route in the controller. We can write out a route for a DELETE request just like we would for a GET request, just by changing the method:

```python
# server/app.py

# imports, config, games, game_by_id, reviews

@app.route('/reviews/<int:id>', methods=['GET', 'DELETE'])
def review_by_id(id):
    review = Review.query.filter(Review.id == id).first()

    if request.method == 'GET':
        review_dict = review.to_dict()

        response = make_response(
            review_dict,
            200
        )

        return response

    elif request.method == 'DELETE':
        db.session.delete(review)
        db.session.commit()

        response_body = {
            "delete_successful": True,
            "message": "Review deleted."
        }

        response = make_response(
            response_body,
            200
        )
```

```
        return response
```

Let's review the new content:

- The `@app.route` decorator accepts `methods` as a default argument. This is simply a list of accepted methods as strings. By default, this list only contains `'GET'`.
- The request context allows us to access the HTTP method used by the request and control flow from there. If the request's method is `GET`, we perform the same actions that we did in the `/games/<int:id>` route. If the method is `DELETE`, we delete the resource.
- Unsupported methods will receive a 405 response code by default. This means "Method Not Allowed".

Now, the question on everyone's minds: how do we actually send a `DELETE` request?

Sending Requests with HTTP Methods Other than `GET`

We all know from experience that we can delete resources on the internet: emails, Instagram posts with unmade beds in the background, etc. There are many different actions that can be carried out over the internet, and browsers are actually very flexible on this front- it's HTML that isn't.

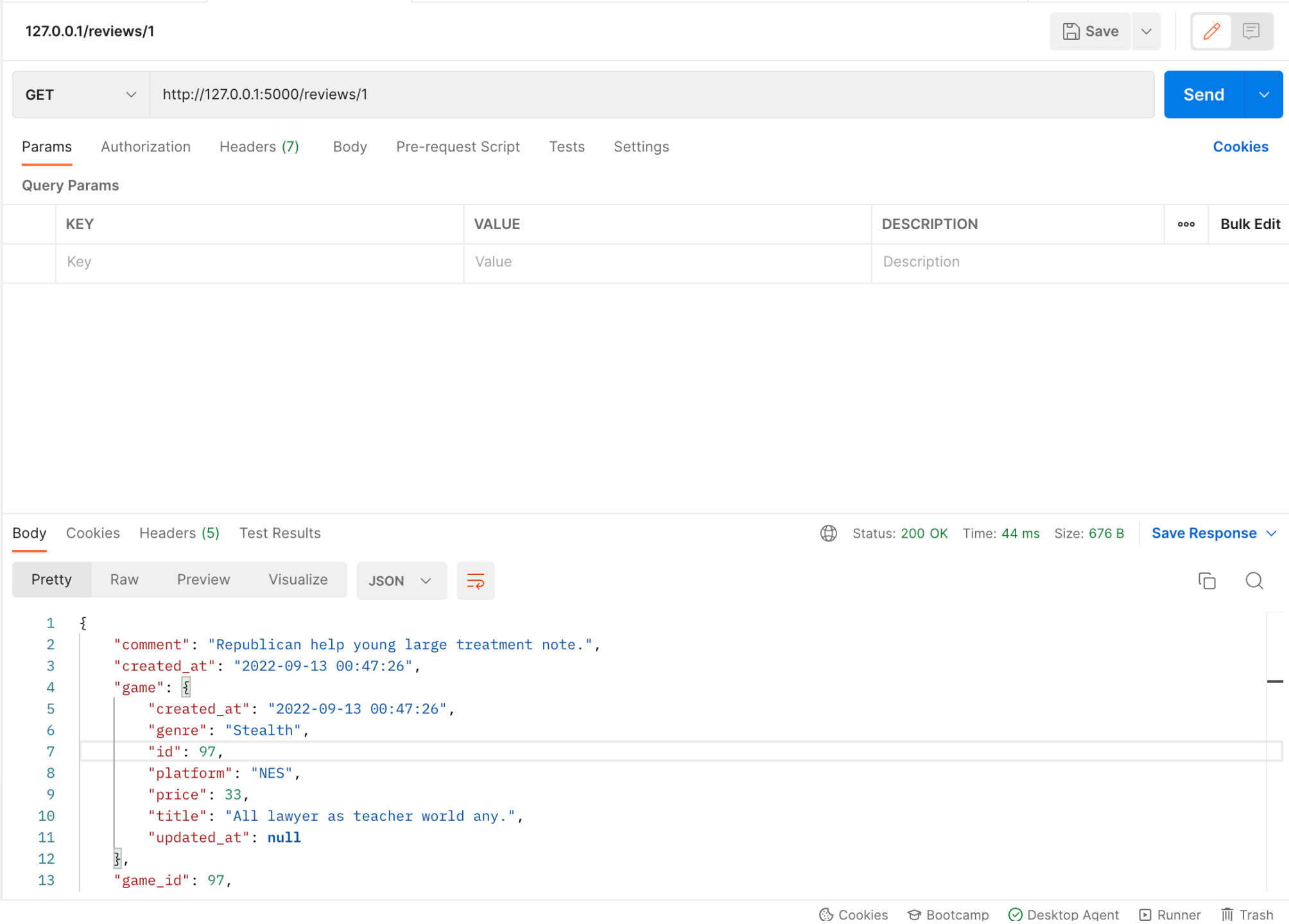
The W3C specifications for HTML say that it should only support `GET` and `POST`. This means that any requests sent to the API through the browser with unsupported methods will have these methods cast to `GET` ...which will then be used to update and delete resources. (We don't want that!)

Because of these limitations, we haven't explored too many HTTP methods with Flask up to this point.

We can use Python scripts and applications with libraries like `requests` to test our `DELETE` resource, but it is often easier to just use Postman.

> **NOTE: Requests to `localhost` or `127.0.0.1` can only be sent using the Postman desktop client. This is unsupported in the cloud.**

Sending a `GET` request (the default method) for the first review will return something similar to the following:


127.0.0.1/reviews/1
Save
GET
http://127.0.0.1:5000/reviews/1
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
KEY
VALUE
DESCRIPTION
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 44 ms
Size: 676 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"comment": "Republican help young large treatment note.",
"created_at": "2022-09-13 00:47:26",
"game": {
"created_at": "2022-09-13 00:47:26",
"genre": "Stealth",
"id": 97,
"platform": "NES",
"price": 33,
"title": "All lawyer as teacher world any.",
"updated_at": null
},
"game_id": 97,
Cookies
Bootcamp
Desktop Agent
Runner
Trash

I know what you're thinking: how could we possibly delete this illuminating review of "All lawyer as teacher world any"? How will anyone know that "Republican help young large treatment note"?

Alas, we need to make sure our API works. Change the request method to `DELETE` with the dropdown menu to the left of the search bar and submit a new request to this resource.


http://127.0.0.1:5001/reviews/1
Save

> **NOTE: You may notice that the port accessed here has switched to 5001. You can run whichever port you like on your machine, as long as there isn't a conflict with another application. We typically use 5555 since it is the easiest 5000s port to remember after 5000, which sees the AirPlay conflict on MacOS.**

Navigate back to the same resource with `GET` and you should see a `500 Internal Server Error`. There's no resource here anymore! This is obviously not the most helpful message, but it can be handled with control flow just like any other error. A simple solution will be included in the solution code at the end of this lesson.

Handling POST Requests

For our next feature, let's give our users the ability to **Create** new reviews. From the frontend, here's how our React component might look:

```
// example code

function ReviewForm({ userId, gameId, onAddReview }) {
  const [comment, setComment] = useState("");
  const [score, setScore] = useState("0");

  function handleSubmit(e) {
    e.preventDefault();
    fetch("http://localhost:9292/reviews", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        comment: comment,
        score: score,
        user_id: userId,
        game_id: gameId,
```

```
        }),
      })
        .then((r) => r.json())
        .then((newReview) => onAddReview(newReview));
    }

    return <form onSubmit={handleSubmit}>{/* controlled form code here*/}</form>;
  }
```

This request is a bit trickier than the last: in order to create a review in the database, we need some way of getting all the data that the user entered into the form. From the code above, you can see that we'll have access to that data in the **body** of the request, as a JSON-formatted string. So in terms of the steps for our server, we need to:

- Handle requests with the `POST` HTTP verb to `/reviews`.
- Access the data in the body of the request.
- Use that data to create a new review in the database.
- Send a response with newly created review as JSON.

Let's start with the easy part. We can create a workflow for a new method like so:

```
#server/app.py

# imports, config, games, game_by_id

@app.route('/reviews', methods=['GET', 'POST'])
def reviews():

    if request.method == 'GET':
        reviews = []
        for review in Review.query.all():
            review_dict = review.to_dict()
            reviews.append(review_dict)
```

```
        response = make_response(
            reviews,
            200
        )

        return response

    elif request.method == 'POST':
        response_body = {}
        response = make_response(
            response_body,
            201
        )

        return response
```

> **NOTE: A 201 status code means that a record has been successfully created.**

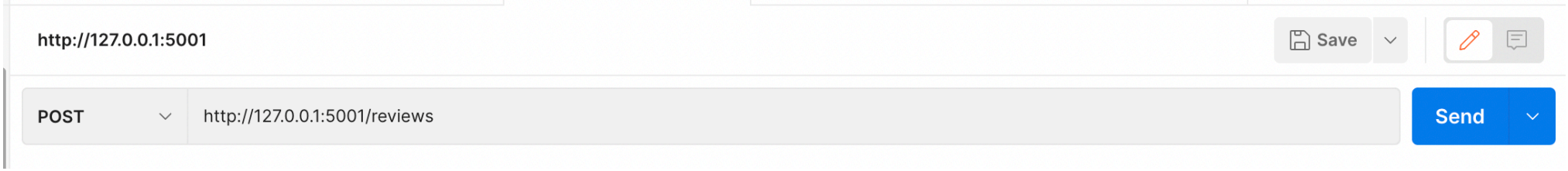
In this new block, we'll need to create a new record using the attributes passed in the request.

```
# server/app.py

# imports, config, games, game_by_id

@app.route('/reviews', methods=['GET', 'POST'])
def reviews():

    if request.method == 'GET':
        reviews = []
        for review in Review.query.all():
            review_dict = review.to_dict()
            reviews.append(review_dict)
```

```
        response = make_response(
            reviews,
            200
        )

        return response

    elif request.method == 'POST':
        new_review = Review(
            score=request.form.get("score"),
            comment=request.form.get("comment"),
            game_id=request.form.get("game_id"),
            user_id=request.form.get("user_id"),
        )

        db.session.add(new_review)
        db.session.commit()

        review_dict = new_review.to_dict()

        response = make_response(
            review_dict,
            201
        )

        return response
```

The request context has access to form data, among many other things. While we haven't created a form here, makeshift forms can still be attached to requests and their attributes can be parsed to create new records. It's important that we create `review_dict` *after* committing the

review to the database, as this populates it with an ID and data from its game and user. Submitting a `POST` request with Postman should return something like this:


http://127.0.0.1:5001
Save
POST
http://127.0.0.1:5001/reviews
Send

Now that we've created, read, and deleted data, let's look at how to update.

Handling PATCH Requests

Onto the last HTTP verb for this unit: `PATCH`! Now that you've learned about `POST` and `DELETE` requests, this should be more straightforward. From the frontend, we might need to use a `PATCH` request to handle a feature that would allow a user to update their review, in case they change their minds:

```
function EditReviewForm({ review, onUpdateReview }) {
  const [comment, setComment] = useState("");
  const [score, setScore] = useState("0");

  function handleSubmit(e) {
    e.preventDefault();
    fetch(`http://localhost:9292/reviews/${review.id}`, {
      method: "PATCH",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        comment: comment,
        score: score,
      }),
    })
      .then((r) => r.json())
      .then((updatedReview) => onUpdateReview(updatedReview));
  }

  return <form onSubmit={handleSubmit}>{/* controlled form code here*/}</form>;
}
```

The steps we'll need to handle on the server for this request are basically a combination of DELETE and POST. We'll need to:

- Handle requests with the `PATCH` HTTP verb to `/reviews/<int:id>`.
- Find the review to update using the ID.
- Access the data in the body of the request.
- Use that data to update the review in the database.
- Send a response with updated review as JSON.

Give it a shot yourself before looking at the solution! You have all the tools you need to get this request working. When you're ready, keep scrolling...

...

...

...

...

...

...

Ok, here's how the code for this route would look:

```
# server/app.py

# imports, config, games, game_by_id, reviews
@app.route('/reviews/<int:id>', methods=['GET', 'PATCH', 'DELETE'])

    # GET

    elif request.method == 'PATCH':
        for attr in request.form:
            setattr(review, attr, request.form.get(attr))
```

```
        db.session.add(review)
        db.session.commit()

        review_dict = review.to_dict()

        response = make_response(
            review_dict,
            200
        )

        return response

    # DELETE
```

- First, we locate the record we want to change.
- Second, we update the record's attributes using `request.form`. We use `setattr()` here because it allows us to use variable values as attribute names- when we don't know which fields are being updated, this is important.
- From that point, this is very similar to our `POST` block from `'/reviews'`. We need to save the updated record to the database and then serve it to the client as JSON.

Run a `PATCH` request in Postman and you should see something similar to the following:

Conclusion

You're at the point now where you can create a JSON API that handles all four CRUD actions: Create, Read, Update, and Delete. With just these four actions, you can build an API for almost any application you can think of!

Solution Code

```
#!/usr/bin/env python3

from flask import Flask, request, make_response
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate

from models import db, User, Review, Game

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False

migrate = Migrate(app, db)

db.init_app(app)

@app.route('/')
def index():
    return "Index for Game/Review/User API"

@app.route('/games')
def games():

    games = []
    for game in Game.query.all():
        game_dict = game.to_dict()
        games.append(game_dict)

    response = make_response(
```

```
            games,
            200
        )

        return response

    @app.route('/games/<int:id>')
    def game_by_id(id):
        game = Game.query.filter(Game.id == id).first()

        game_dict = game.to_dict()

        response = make_response(
            game_dict,
            200
        )

        return response

    @app.route('/reviews', methods=['GET', 'POST'])
    def reviews():

        if request.method == 'GET':
            reviews = []
            for review in Review.query.all():
                review_dict = review.to_dict()
                reviews.append(review_dict)

            response = make_response(
                reviews,
                200
            )
```

```
        return response

    elif request.method == 'POST':
        new_review = Review(
            score=request.form.get("score"),
            comment=request.form.get("comment"),
            game_id=request.form.get("game_id"),
            user_id=request.form.get("user_id"),
        )

        db.session.add(new_review)
        db.session.commit()

        review_dict = new_review.to_dict()

        response = make_response(
            review_dict,
            201
        )

        return response

@app.route('/reviews/<int:id>', methods=['GET', 'PATCH', 'DELETE'])
def review_by_id(id):
    review = Review.query.filter(Review.id == id).first()

    if review == None:
        response_body = {
            "message": "This record does not exist in our database. Please try again."
        }
        response = make_response(response_body, 404)

        return response
```

```
    else:
        if request.method == 'GET':
            review_dict = review.to_dict()

            response = make_response(
                review_dict,
                200
            )

            return response

        elif request.method == 'PATCH':
            for attr in request.form:
                setattr(review, attr, request.form.get(attr))

            db.session.add(review)
            db.session.commit()

            review_dict = review.to_dict()

            response = make_response(
                review_dict,
                200
            )

            return response

        elif request.method == 'DELETE':
            db.session.delete(review)
            db.session.commit()

            response_body = {
```

```
            "delete_successful": True,
            "message": "Review deleted."
        }

        response = make_response(
            response_body,
            200
        )

        return response

@app.route('/users')
def users():

    users = []
    for user in User.query.all():
        user_dict = user.to_dict()
        users.append(user_dict)

    response = make_response(
        users,
        200
    )

    return response

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Resources

- Flask - Pallets (https://flask.palletsprojects.com/en/2.2.x/)
- POST - Mozilla (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST)
- PATCH - Mozilla (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/PATCH)
- DELETE - Mozilla (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/DELETE)
- flask.json.jsonify Example Code - Full Stack Python (https://www.fullstackpython.com/flask-json-jsonify-examples.html)
- SQLAlchemy-serializer - PyPI (https://pypi.org/project/SQLAlchemy-serializer/)

This tool needs to be loaded in a new browser window

Load Building a POST/PATCH/DELETE API (CodeGrade) in a new window